

Série TP 6 : Triggers

Corrigé

Exercice 1 :

Soit une base de données dont le schéma est le suivant :

Table	Colonne	Type	Description
EMPLOYE	N_EMP	VARCHAR2(4)	Numéro employé
	NOM_EMP	VARCHAR2(20)	Nom employé
	QUALIF_EMP	VARCHAR2(12)	Qualification employé
TRANSPORTER	N_TRANSPORTE	VARCHAR2(5)	Numéro transport
	N_VISITE	VARCHAR2(5)	Numéro visite
	N_EMPLOYE	VARCHAR2(4)	Numéro employé
CHANTIER	N_CHANTIER	VARCHAR2(10)	Numéro chantier
	NOM_CH	VARCHAR2(10)	Nom chantier
	ADRESSE_CH	VARCHAR2(15)	Adresse chantier
VEHICULE	N_VÉHICULE	VARCHAR2(10)	Numéro véhicule
	TYPE_VÉHICULE	VARCHAR2(11)	Type véhicule
	KILOMETRAGE	NUMBER	Kilométrage véhicule
VISITE	N_VISITE	VARCHAR2(5)	Numéro visite
	N_CHANTIER	VARCHAR2(10)	Numéro chantier
	N_VÉHICULE	VARCHAR2(10)	Numéro véhicule
	KILOMETRES	NUMBER	Kilomètres parcourus
	DATE_JOUR	DATE	Date de la visite
	N_CONDUCTEUR	VARCHAR2(4)	Numéro conducteur

Travail à faire :

Ecrire le déclencheur 'TrigConducteur' sur la table 'TRANSPORTER' permettant de vérifier qu'à chaque nouveau transport, le passager déclaré n'est pas déjà enregistré en tant que conducteur le même jour.

Si c'est le cas, l'insertion dans la table 'TRANSPORTER' génère une erreur qui est lancée par RAISE_APPLICATION_ERROR(-20000, 'Erreur : Cet employé est en mission ce jour') ;

```
CREATE OR REPLACE TRIGGER TrigConducteur
BEFORE INSERT ON TRANSPORTER
FOR EACH ROW
DECLARE
    v_conducteur_existe NUMBER;
    v_date_visite VISITE.DATE_JOUR%TYPE;
BEGIN
    SELECT DATE_JOUR INTO v_date_visite
    FROM VISITE
    WHERE N_VISITE = :NEW.N_VISITE;

    SELECT COUNT(*) INTO v_conducteur_existe
    FROM VISITE v
    WHERE v.N_CONDUCTEUR = :NEW.N_EMPLOYE
    AND v.DATE_JOUR = v_date_visite;

    IF v_conducteur_existe > 0 THEN
        RAISE_APPLICATION_ERROR(-20000, 'Erreur : Cet employé est en mission ce jour');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Erreur : Visite non trouvée');
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002, 'Erreur inattendue : ' || SQLERRM);
END;
/
```

Exercice 2 :

Soit une base de données dont le schéma est le suivant (les clés primaires des relations sont soulignées).

- Employé (NumEmp NUMBER, NomEmp VARCHAR(20), PrenomEmp VARCHAR(20), FonctionEmp VARCHAR(20), SalaireEmp NUMBER, VilleEmp VARCHAR(20))
- GradeSalaire(Grade VARCHAR(20), SalaireMin NUMBER, SalaireMax NUMBER)
- Département (NumDep NUMBER, NomDep VARCHAR(20), BudgetDep NUMBER, VilleDep VARCHAR(20))
- DirigerDepartement (#NumDep NUMBER, #NumEmp NUMBER, DateDebut DATE, DateFin DATE)

- Projet (**NumProjet** NUMBER, TitreProjet VARCHAR(100), Montant NUMBER, DateDebut DATE, DateFin DATE)
- Departement_Projet (**#NumProjet** NUMBER, **#NumDep** NUMBER, MontantParticipation NUMBER)

Travail à faire :

Ecrire l'ensemble des déclencheurs pour assurer les contraintes suivantes :

1. Vérifier que l'employé directeur d'un département existe bien, et que
DirigerDepartement.DateDebut < DirigerDepartement.DateFin

```
CREATE OR REPLACE TRIGGER TrigVerifDirecteur
BEFORE INSERT OR UPDATE ON DirigerDepartement
FOR EACH ROW
DECLARE
    v_employe_existe NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_employe_existe
    FROM Employe
    WHERE NumEmp = :NEW.NumEmp;

    IF v_employe_existe = 0 THEN
        RAISE_APPLICATION_ERROR(-20010, 'Erreur : Employé directeur ' || :NEW.NumEmp || '
n"existe pas');
    END IF;

    IF :NEW.DateDebut >= :NEW.DateFin THEN
        RAISE_APPLICATION_ERROR(-20011, 'Erreur : DateDebut doit être antérieure à DateFin');
    END IF;

END;
/
```

2. La somme des montants de participation aux projets d'un département ne dépasse pas le budget de ce dernier.

```
CREATE OR REPLACE TRIGGER TrigVerifBudget
BEFORE INSERT OR UPDATE ON Departement_Projet
FOR EACH ROW
DECLARE
    v_budget_departement Departement.BudgetDep%TYPE;
    v_total_participation NUMBER;
```

```
BEGIN
SELECT BudgetDep INTO v_budget_departement
FROM Departement
WHERE NumDep = :NEW.NumDep;

SELECT NVL(SUM(MontantParticipation), 0)
INTO v_total_participation
FROM Departement_Projet
WHERE NumDep = :NEW.NumDep
AND (NumProjet != :NEW.NumProjet OR INSERTING);

v_total_participation := v_total_participation + :NEW.MontantParticipation;

IF v_total_participation > v_budget_departement THEN
    RAISE_APPLICATION_ERROR(-20012,
        'Erreur : Budget département ' || :NEW.NumDep || ' insuffisant. ' ||
        'Budget: ' || v_budget_departement ||
        ', Total participations: ' || v_total_participation);
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20013, 'Erreur : Département ' || :NEW.NumDep || ' non trouvé');
END;
```

/